

Elementare Datentypen und Referenzen – Teil 2

Referenzen und Arrays

Programmierung 1

PIB-PR1 / DFIW-PRG1

Software Engineering und Software Quality Assurance { SESQA }

Prof. Dr. Martin Burger { 13. November 2025 }

noan



Plan der Präsenzveranstaltungen

VW	KW	 Inhalte in der „Vorlesung“	Mo	Di	Mi	Do
1	42	Willkommen und erste Orientierung, Lebenslanges Lernen				
2	43	Einstieg in Java (Kap. 1)				
3	44	Teamarbeit				
4	45	Klassen und Objekte (Kap. 2)				
5	46	Elementare Datentypen und Referenzen (Kap. 3)				
6	47	Methoden benutzen Instanzvariablen (Kap. 4)				
7	48	Ein Programm schreiben (Kap. 5)				
8	49	Die Java-API kennenlernen (Kap. 6)				
9	50	Vererbung und Polymorphie (Kap. 7)				
10	51	Interfaces und abstrakte Klassen (Kap. 8)				
	52	Vorlesungsfreie Zeit (Jahreswechsel)				
	01	Vorlesungsfreie Zeit (Jahreswechsel)				
11	02	Konstruktoren und Garbage Collection (Kap. 9)				
12	03	Zahlen und Statisches (Kap. 10)				
13	04	Exception-Handling (Kap. 13)				
14	05	Serialisierung und Datei-I/O (Kap. 16)				
15	06	Fragen und Antworten (Puffer)				

„Kein Plan 
überlebt den
ersten
Kontakt  mit
der Realität .

Frei nach Helmuth Karl
Bernhard von Moltke

Elementare Datentypen und Referenzen

 Plan für diese Woche

 Typsicherheit

 Variablendeklaration

 Elementartypen

 Literale Werte

 Schlüsselwörter

 Pair Programming

 Objektreferenzen

 Objekte auf dem Heap

 Arrays

 Übungen

 Pair Programming

 Spaß

📖 Elementare Datentypen und Referenzen

🔗 Verbindungen herstellen

👤 Tauschen Sie sich mit einer Nachbar:in aus!

🔧 Was haben Referenzen auf Objekte und Fernbedienungen gemeinsam?

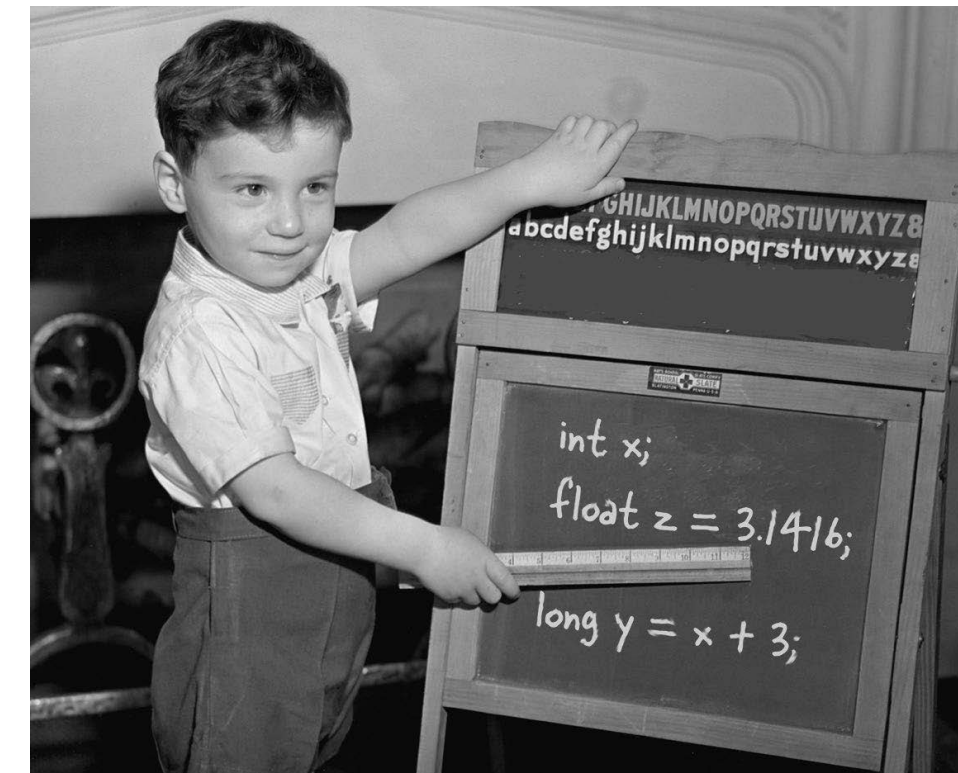
♻️ Was lebt auf dem Garbage Collectible Heap?

🗄️ Was haben ein Array und ein Schrank voller Tassen gemeinsam?

🙋 Welche Fragen haben Sie zu diesem Kapitel?

3 Elementare Datentypen und Referenzen

Verstehen Sie Ihre Variablen



Variablen gibt es in zwei Geschmacksrichtungen: **elementare (oder auch primitive) Typen und Referenztypen**. Bis jetzt haben Sie Variablen an zwei Orten verwendet – als Objekt-Zustand (Instanzvariablen) und als **lokale** Variablen (Variablen, die innerhalb einer *Methode* deklariert werden).

Später werden wir Variablen als Argumente (Werte, die *von* aufrufendem Code an eine Methode übergeben werden) und als Rückgabewerte (Werte, die aus einer Methode *an* aufrufenden Code zurückgeliefert werden) nutzen. Wir haben Variablen bisher als einfache, elementare Integer-Werte (Typ `int`) deklariert, Sie haben jedoch auch Variablen gesehen, die als etwas Komplexeres deklariert wurden, zum Beispiel als String oder Array. **Aber das Leben kann doch nicht nur aus Integer-Werten, Strings und Arrays bestehen!** Wie wäre es beispielsweise mit einem `HaustierBesitzer`-Objekt und einer Instanzvariablen vom Typ `Hund`? Oder mit einem `Auto`-Objekt, das einen Motor hat? In diesem Kapitel werden wir die Geheimnisse der Java-Typen lüften (zum Beispiel das Geheimnis des Unterschieds zwischen Elementartypen und Referenzen). Wir werden uns ansehen, was man alles als Variablen deklarieren kann, was Sie in einer Variablen speichern und was Sie mit einer Variablen tun können. Zum Schluss lernen Sie das Leben auf dem Garbage Collectible Heap hautnah kennen.



Objektreferenzen

Elementare Variablen

🕒 Objektreferenzen

- Variablen von einem elementaren Typ wie zum Beispiel `boolean` oder `int` werden als **elementare Variablen** bezeichnet.
- Elementare Variablen enthalten direkt **den Wert** des entsprechenden Datentyps, also beispielsweise `true` oder `42`.



Referenzvariablen

🖱️ Objektreferenzen

- Variablen, die ein Objekt „enthalten“, gibt es in diesem Sinne nicht. Sie **zeigen** auf Objekte.
- Variablen, die auf Objekte zeigen, werden **Referenzvariablen** genannt.
- Referenzvariablen enthalten nicht die eigentlichen Daten des Objekts, sondern eine **Referenz** oder einen „Zeiger“ auf die Speicheradresse, an der sich das Objekt befindet.



Zugriff auf Objekte

🖱️ Objektreferenzen

- Die Java Virtual Machine (JVM) weiß, wie sie **über die Referenz** auf das Objekt zugreifen kann, um auf die darin enthaltenen Daten zuzugreifen oder die Methoden des Objekts aufzurufen.
- Dies ermöglicht eine effiziente Speicherverwaltung, da die JVM den Speicherplatz für die eigentlichen Daten des Objekts **dynamisch** zuweisen und freigeben kann. 😊
Stichwort: Garbage Collection (GC)



In anderen Sprachen wie C++ ist das Ihr Job! 😱

Punktoperator

🕒 Objektreferenzen

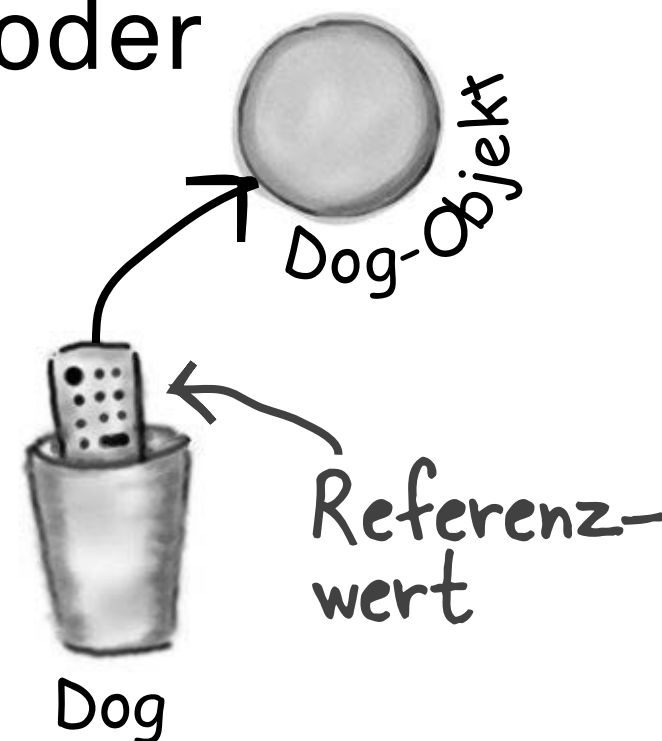
- Sie verwenden den **Punktoperator** auf einer Referenzvariablen, um auf die Daten des Objekts zuzugreifen oder seine Methoden aufzurufen.
- Sie sagen der JVM dadurch: „Benutze das Ding vor dem Punkt, um das Ding nach dem Punkt zu erhalten.“
- Zum Beispiel bedeutet `myDog.bark()`:
„Benutze das von der Variablen `myDog` referenzierte Objekt, um seine Methode `bark()` aufzurufen.“



Objekt-Fernbedienung

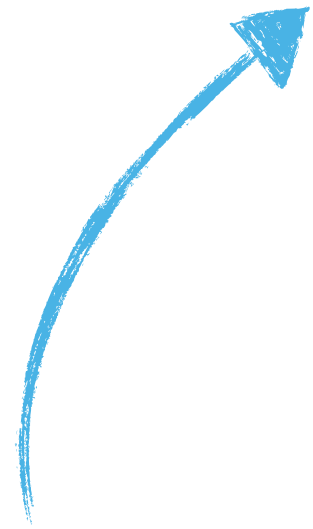
📺 Objektreferenzen

- Eine Referenzvariable funktioniert wie eine „**Fernbedienung**“ für das referenzierte Objekt.
- Die Verwendung des Punktoperators auf einer Referenzvariablen entspricht dem Drücken eines Knopfes auf dieser Fernbedienung.
- Sie können so sozusagen „auf Knopfdruck“ auf eine Methode oder Instanzvariable zuzugreifen.
- Im „Becher“ landet diesmal die Fernbedienung.




```
Dog myDog = new Dog();
```

1.



3.



2.



Deklaration, Erstellung und Zuweisung von Objekten

Eine Referenzvariable deklarieren

 Objektreferenzen

```
Dog myDog = new Dog();
```

1. 

Deklaration, Erstellung und Zuweisung von Objekten

- Weist die JVM an, Speicherplatz für eine Referenzvariable zuzuweisen.
- Die Variable erhält den Namen myDog.
- Die Referenzvariable hat dauerhaft den Typ Dog. → Typsicherheit
- Die „Fernbedienung“ hat passende Knöpfe für Hunde, aber nicht für Autos, Bücher usw.

Ein Objekt erstellen

 Objektreferenzen

```
Dog myDog = new Dog();
```

2.



Deklaration, **Erstellung** und Zuweisung von Objekten

- Weist die JVM an, Speicherplatz auf dem Heap für ein neues Dog-Objekt zuzuweisen.
- Dies werden wir im Kapitel „Konstruktoren und Garbage Collection“ genauer betrachten.

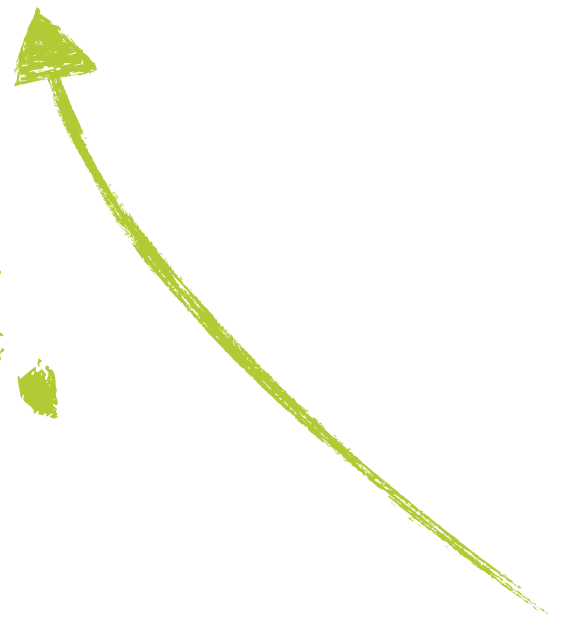
Objekt und Referenz verknüpfen



Objektreferenzen

```
Dog myDog = new Dog();
```

3.



Deklaration, Erstellung und **Zuweisung** von Objekten

- Weist der Variablen myDog das neue Objekt Dog zu.
- Mit anderen Worten, „programmiert“ die Fernbedienung so, dass sie mit genau diesem Exemplar von Hund verbunden ist.

! Quiz

🔊 Objektreferenzen

- 🤔 Ich mache einige Aussagen – sind diese Aussagen richtig oder falsch?
- 👍 Bei **richtigen** Aussagen **stehen** Sie **kurz auf**!
- 👎 Bei **falschen** Aussagen **bleiben** Sie **sitzen**!



Variablen eines elementaren Typs wie `char` oder `float` werden als elementare Variablen bezeichnet.

Richtig!

Elementare Variablen enthalten direkt den Wert des entsprechenden Datentyps, also beispielsweise "Dies ist ein String" oder 2147483647.

Nur wenn es sich um primitive Typen und Daten handelt. Zur Erinnerung: Ein String ist ein Objekt.

Referenzvariablen enthalten die Daten des referenzierten Objekts.

Variablen, die ein Objekt „enthalten“, gibt es in diesem Sinne nicht. Referenzvariablen „zeigen“ auf Objekte.

Referenzvariablen enthalten eine Referenz
oder einen „Zeiger“ auf die Speicheradresse,
an der sich das Objekt befindet.

Richtig!

Der Wert einer Referenzvariablen sind die Bits, die eine Zugriffsmöglichkeit auf ein Objekt auf dem Heap repräsentieren.

Richtig!

Eine Referenzvariable funktioniert wie eine „Fernbedienung“ für das referenzierte Objekt: Sie können sozusagen „auf Knopfdruck“ den Punktoperator „auslösen“, um auf die Daten des Objekts zuzugreifen oder seine Methoden aufzurufen.

Richtig!

`Dog myDog = new Dog()` bedeutet in zeitlicher Reihenfolge: 1. Deklaration, dann 2. Zuweisung und schließlich 3. Erstellung des Objekts.

1. Deklaration, 2. Erstellung und 3. Zuweisung

 Welcher **eine** Gedanke zu
„ Objektreferenzen“ war für
Sie am wichtigsten?
 Schreiben Sie ihn auf!



Durchatmen und  Sortieren

Objekte auf dem Heap

**Jetzt die Seiten 57
und 58
wiederholen!**



Sie haben dafür inklusive Pause 10 Minuten Zeit.

Das Leben auf dem Garbage Collectible Heap

```
Book b = new Book();
```

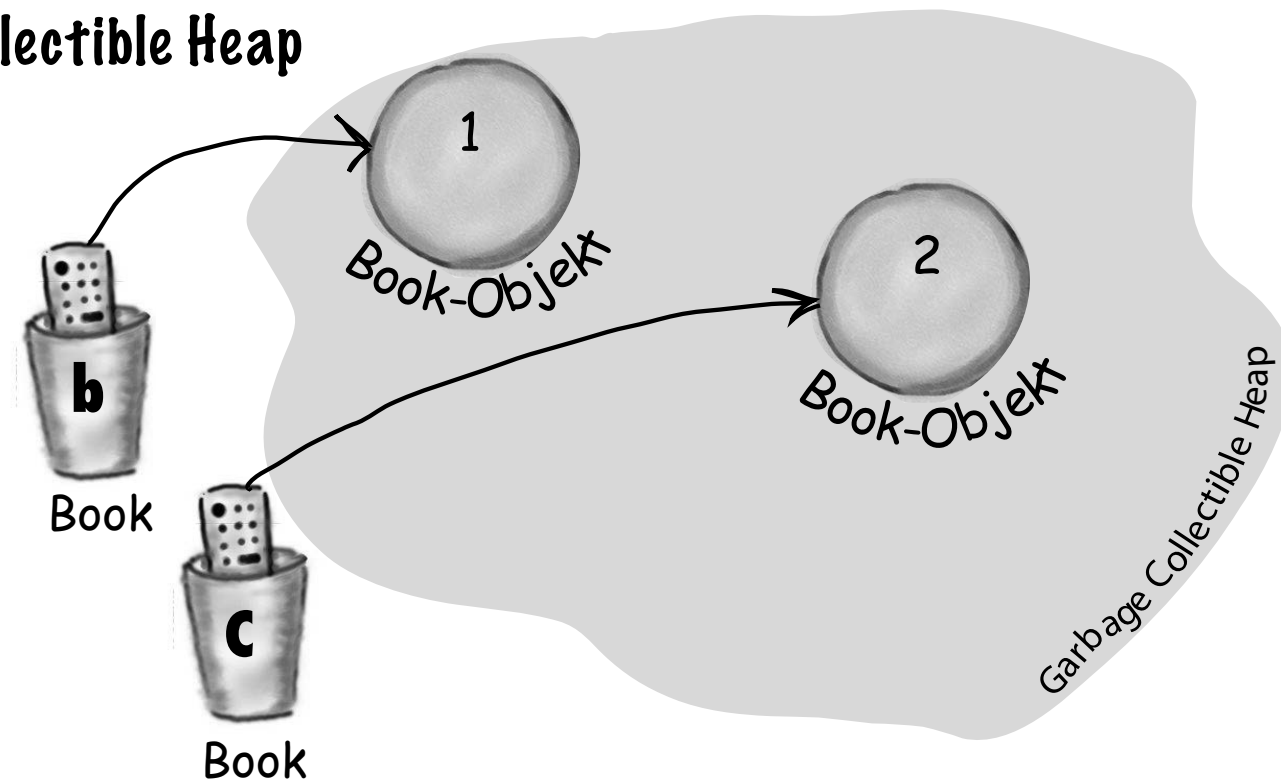
```
Book c = new Book();
```

Deklariert zwei Book-Referenzvariablen. Erstellt zwei neue Book-Objekte. Weist die Book-Objekte den Referenzvariablen zu.

Die zwei Book-Objekte leben nun auf dem Heap.

Referenzen: 2

Objekte: 2



```
Book d = c;
```

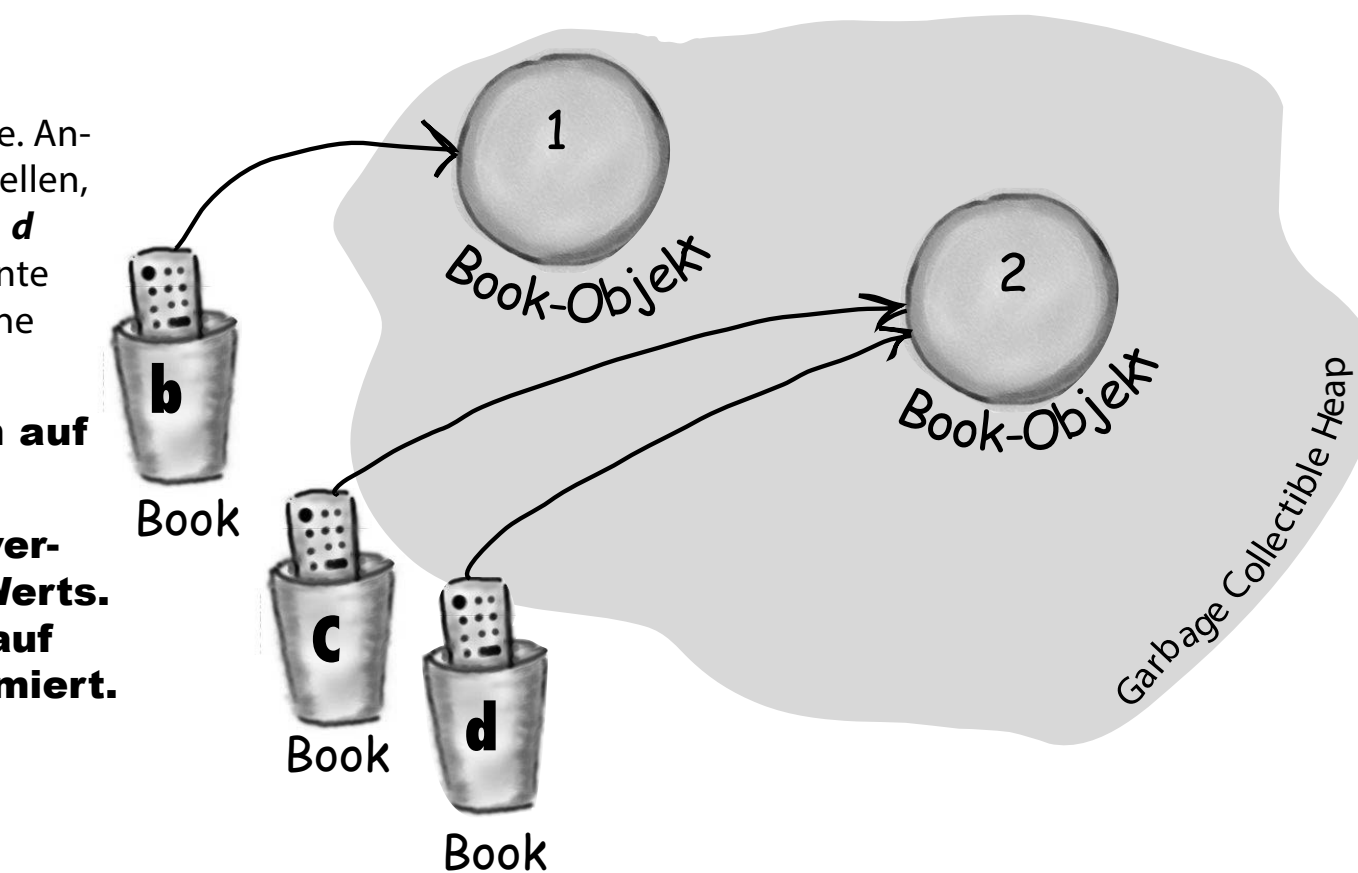
Deklariert eine neue Book-Referenzvariable. Anstatt ein neues, drittes Book-Objekt zu erstellen, wird der Wert der Variablen **c** der Variablen **d** zugewiesen. Aber was heißt das? Man könnte auch sagen: »Nimm die Bits in **c**, mache eine Kopie davon und packe diese Kopie in **d**.«

Die Variablen c und d verweisen auf das gleiche Objekt.

Die Variablen c und d enthalten verschiedene Kopien des gleichen Werts. Beide Fernbedienungen wurden auf den gleichen Fernseher programmiert.

Referenzen: 3

Objekte: 2



```
c = b;
```

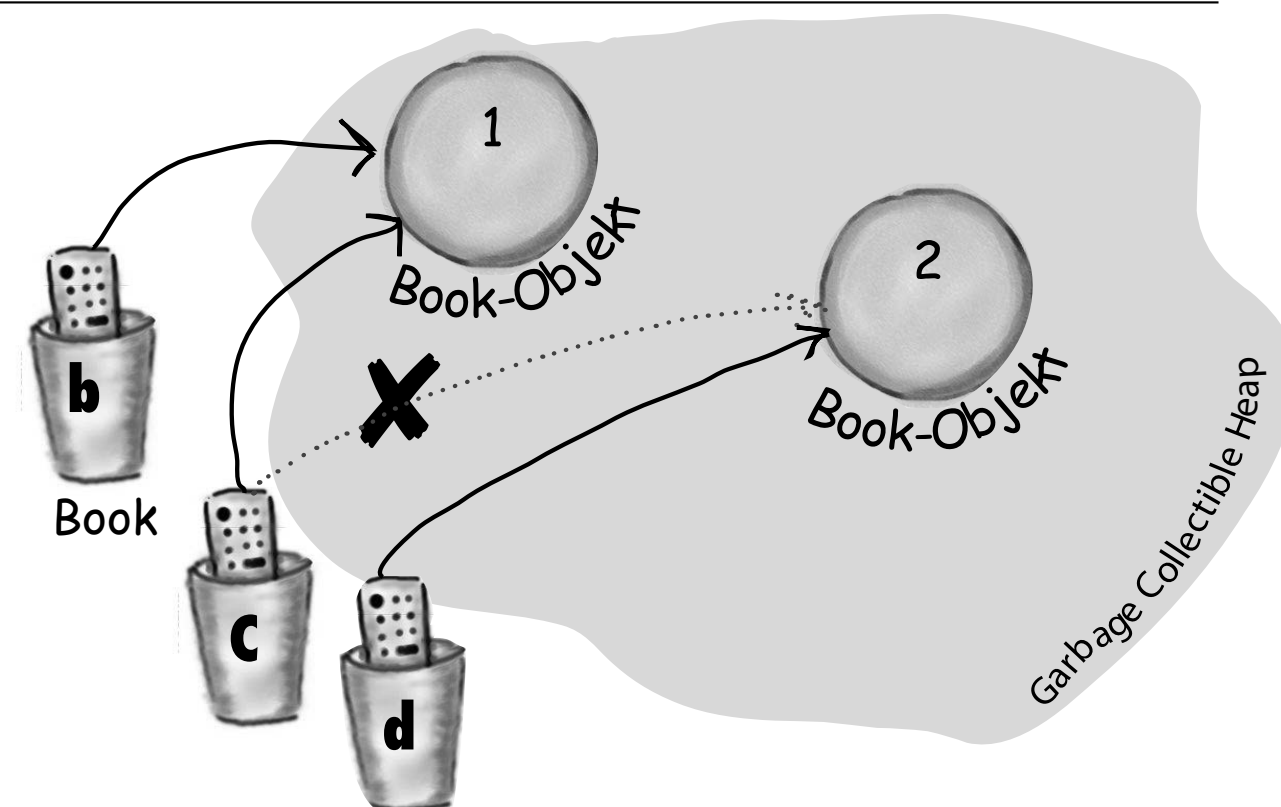
Weist den Wert der Variablen **b** der Variablen **c** zu. Inzwischen wissen Sie, was das heißt. Die Bits in **b** werden kopiert, und die Kopie wird in **c** gespeichert.

Sowohl b wie auch c verweisen auf das gleiche Objekt.

Die Variable c verweist nicht länger auf ihr altes Book-Objekt.

Referenzen: 3

Objekte: 2



Das Leben und Sterben auf dem Heap

```
Book b = new Book();
```

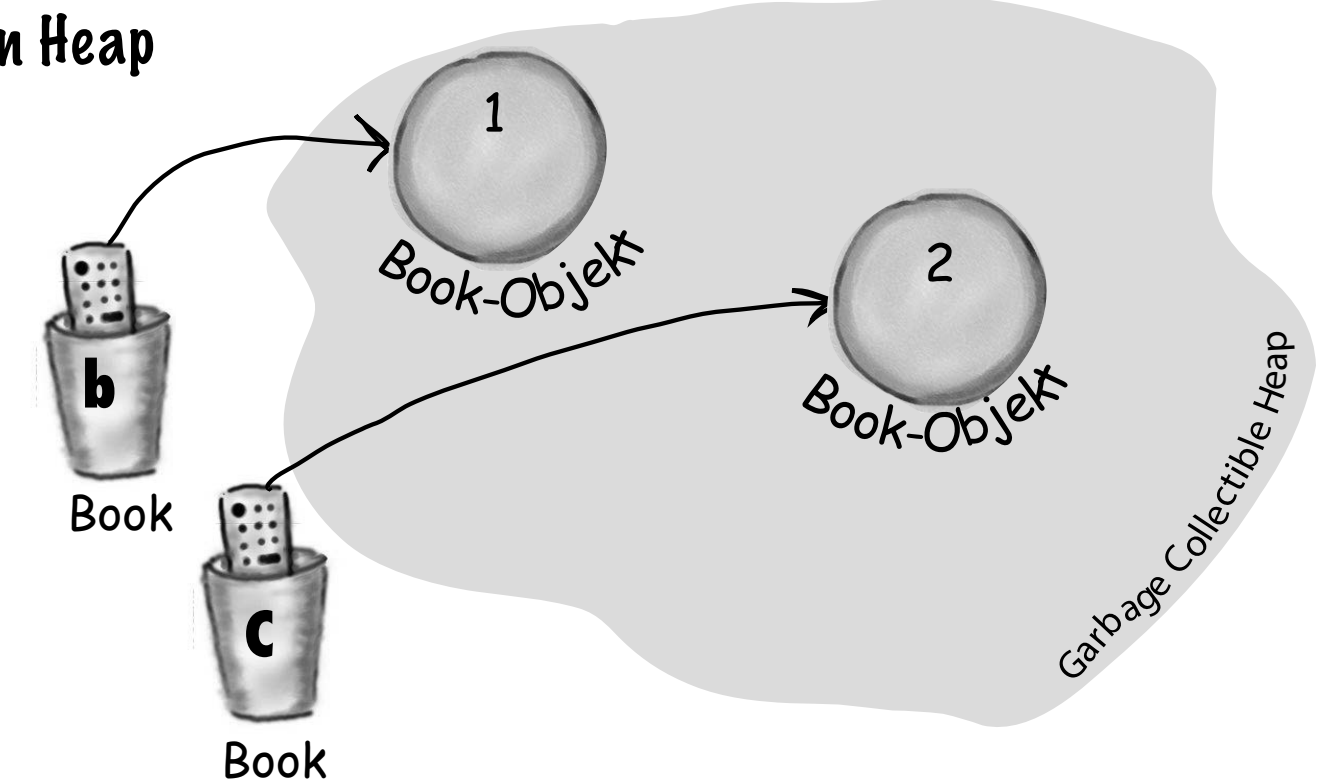
```
Book c = new Book();
```

Deklariert zwei Book-Referenzvariablen. Erstellt zwei neue Book-Objekte. Weist die Book-Objekte den Referenzvariablen zu.

Die zwei Book-Objekte leben nun auf dem Heap.

Aktive Referenzen: 2

Erreichbare Objekte: 2



```
b = c;
```

Weist der Variablen **b** den Wert von **c** zu. Die Bits in der Variablen **c** werden kopiert, die neue Kopie wird in **b** gepackt. Beide Variablen enthalten identische Werte.

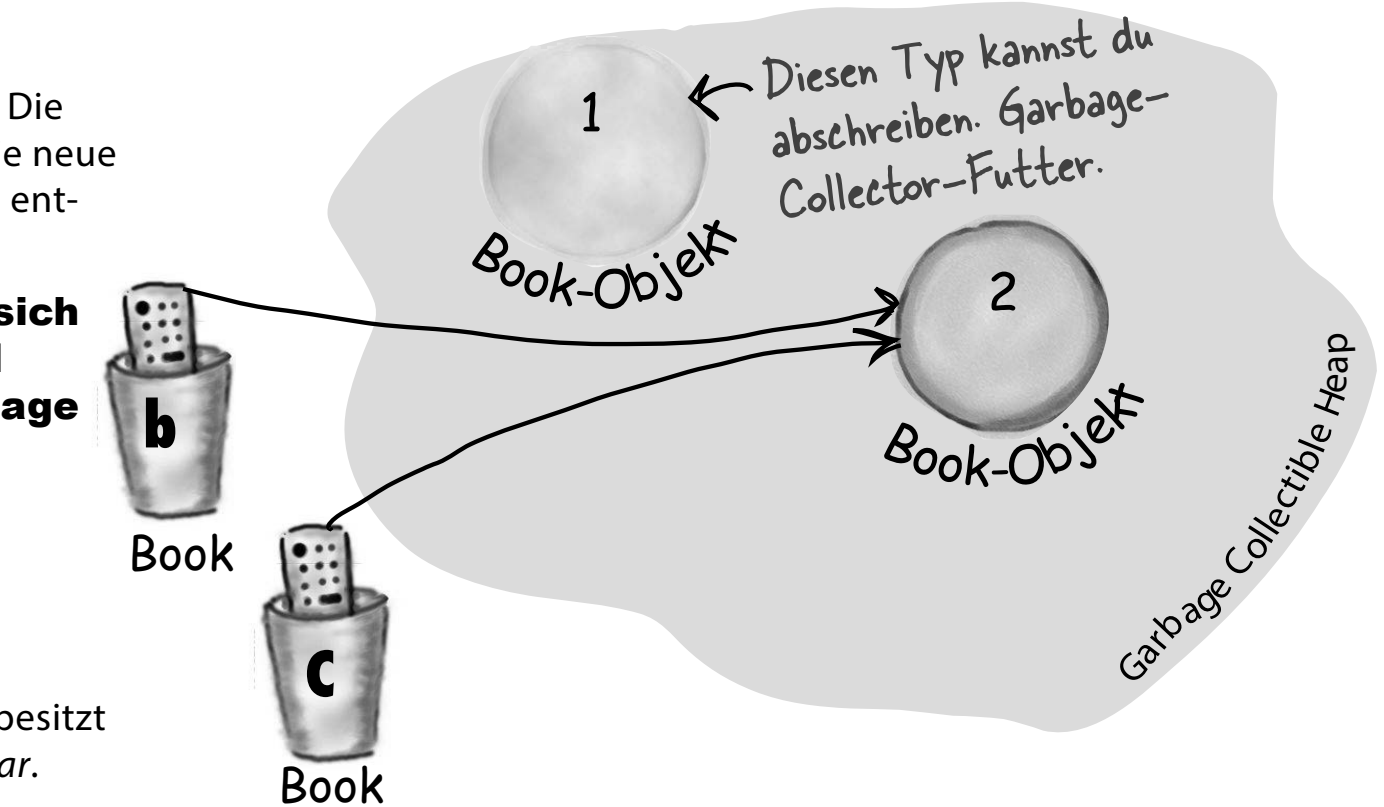
Die Variablen b und c beziehen sich auf das gleiche Objekt. Objekt 1 wurde aufgegeben und zur Garbage Collection (GC) freigegeben.

Aktive Referenzen: 2

Erreichbare Objekte: 1

Aufgegebene Objekte: 1

Das erste von **b** referenzierte Objekt (1) besitzt keine Referenzen mehr. Es ist *unerreichbar*.



```
c = null;
```

Weist der Variablen **c** den Wert **null** zu. Dadurch wird **c** zu einer *null-Referenz*. Das heißt, sie verweist auf nichts. Aber sie ist immer noch eine Referenzvariable, der immer noch ein anderes Book-Objekt zugewiesen werden kann.

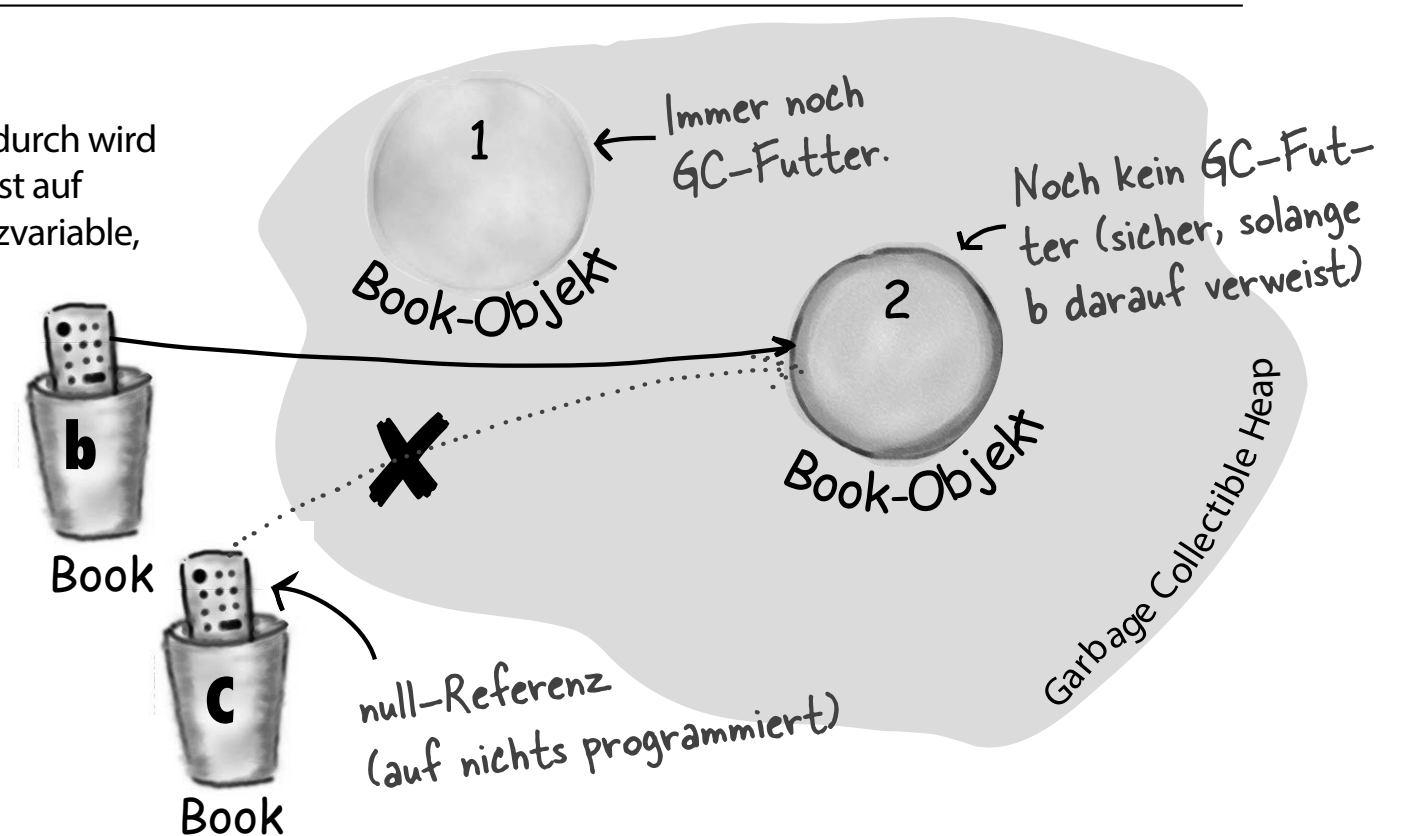
Objekt 2 enthält immer noch eine aktive Referenz (b). Solange das der Fall ist, wird das Objekt nicht für die GC freigegeben.

Aktive Referenzen: 1

null-Referenzen: 1

Erreichbare Objekte: 1

Aufgegebene Objekte: 1



! ? Quiz

♻️ Objekte auf dem Heap

- 🤔 Ich mache einige Aussagen – sind diese Aussagen richtig oder falsch?
- 👍 Bei **richtigen** Aussagen **stehen** Sie **kurz auf**!
- 👎 Bei **falschen** Aussagen **bleiben** Sie **sitzen**!




```
Car a = new Car();
```

```
Car b = new Car();
```

→ Zwei Car-Objekte leben nun auf dem Heap.

Richtig!

```
Car a = new Car();
```

```
Car b = new Car();
```

```
Car c = a;
```

→ Es gibt nun 3 Referenzen auf 2 Objekte. Die Variablen b und c verweisen auf das selbe Objekt.

Die Variablen a und c verweisen auf das selbe Objekt.

Wenn eine Referenzvariable kein Objekt referenziert, hat sie den Wert null.

Richtig!


```
Car a = new Car(); Car b = new Car(); a = b;
```

→ Die Variablen a und b beziehen sich auf das selbe Objekt.
Das 1. Car-Objekt wurde aufgegeben und zur Garbage
Collection (GC) freigegeben.

Richtig!

```
Car a = new Car(); Car b = new Car(); a = null;
```

→ Weist der Variablen a den Wert null zu. Dadurch wird a zu einer null-Referenz. Das heißt, a wurde zur Garbage Collection (GC) freigegeben.

Das heißt, a verweist aktuell auf nichts. Nur Objekte können zur GC freigegeben werden.

 Welcher **eine** Gedanke zu
„ Objekte auf dem Heap“ war
für Sie am wichtigsten?
 Schreiben Sie ihn auf!



Durchatmen und  Sortieren



Arrays

Eine Art Spindsystem...

Arrays

- Spoiler-Alarm: Java bietet eine Vielzahl unterschiedlicher, hochoptimierter Datenstrukturen wie Maps, Trees und Sets.
 - Java Collections in Programmierung 2
- Arrays bieten eine schnelle, effiziente und **geordnete Liste** von Dingen „gleicher Art“.
 - Mit anderen Worten: ... von Dingen, die in den „Becher“ des gleichen Typs passen.
- Über die **Indexposition** können Sie auf beliebige Elemente (Objekte) zugreifen.
 - Position: 0 bis „Länge - 1“



... voller Becher.

Arrays

- Jedes Element eines Arrays ist eine **Variable vom selben Typ**:
 - entweder Elementartypen (boolean, int, ...)
 - oder Objektreferenzen (Car, Dog, ...).
- Alles, was in den „Becher“ dieses Typs passt, passt auch in jedes Array-Element.
 - In einem Array vom Typ `int` (`int[]`) kann jedes Element eine Ganzzahl (`int`) enthalten.
 - In einem Dog-Array (`Dog[]`) kann jedes Element eine Hunde-**Fernbedienung** enthalten.
- Jedes Array ist selbst ein **Objekt**, unabhängig davon, ob die Elemente elementare Typen oder Objektreferenzen sind.




```
1 import java.util.*;
2
3 public class LockerWithPrimitivesDemo {
4     public static void main(String[] args) {
5         // Declare an int array variable.
6         // An array variable is a remote control for an array object.
7         int[] numbers;
8
9         // Create a new int array of length 4 and assign it to the
10        // previously declared variable named numbers.
11        numbers = new int[4];
12
13        // Assign an int value to each element in the array.
14        // A quick reminder: The elements in an int array are simply
15        // numbered int variables.
16        numbers[0] = 11;
17        numbers[1] = 47;
18        numbers[2] = 8;
19        numbers[3] = 15;
20
21        // Hint: You can use the Arrays.toString() method to convert an
22        // array into a human-readable string.
23        System.out.println(Arrays.toString(numbers));
24
25        // Just like any other variable, you can change the values in the
26        // array.
27        numbers[3] = 42;
28        System.out.println(Arrays.toString(numbers));
29
30        // What would happen if we were to go beyond the end of the array?
31        // numbers[4] = 313;
32    }
33 }
34
```

STDIN

Input for the program (Optional)

Output:

[11, 47, 8, 15]

[11, 47, 8, 42]

Jetzt Seite 60 wiederholen!



Sie haben dafür inklusive Pause 5 Minuten Zeit.

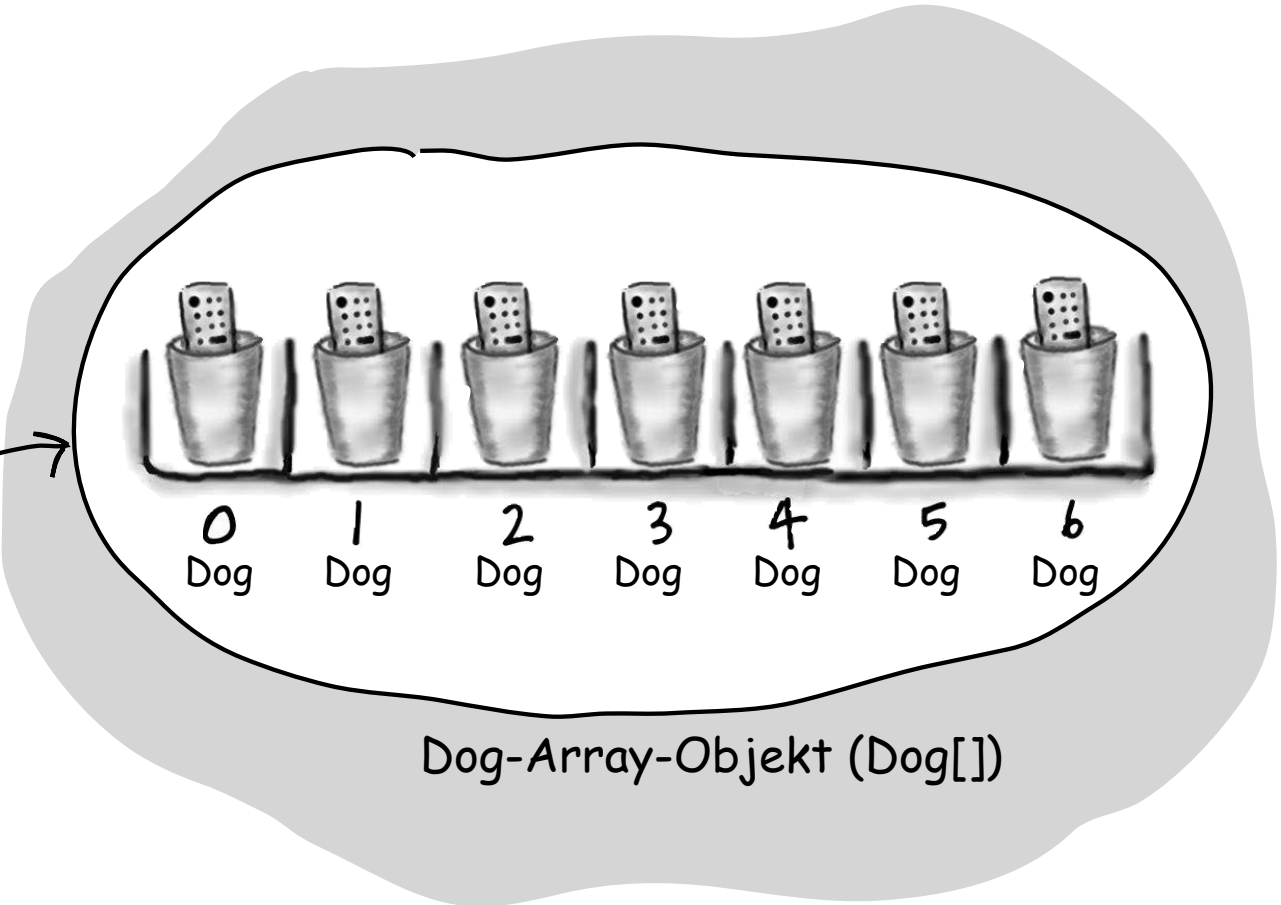
Ein Array aus Dog-Objekten erstellen

- 1 Eine Dog-Array-Variable deklarieren.
`Dog[] pets;`

- 2 Ein neues Dog-Array mit 7 Elementen erstellen und es der zuvor deklarierten `Dog[]`-Variablen `pets` zuweisen.
`pets = new Dog[7];`

Was fehlt hier?

Hunde (Dog-Objekte)! Wir haben ein Array mit Dog-Referenzen, aber keine wirklichen Dog-Objekte!



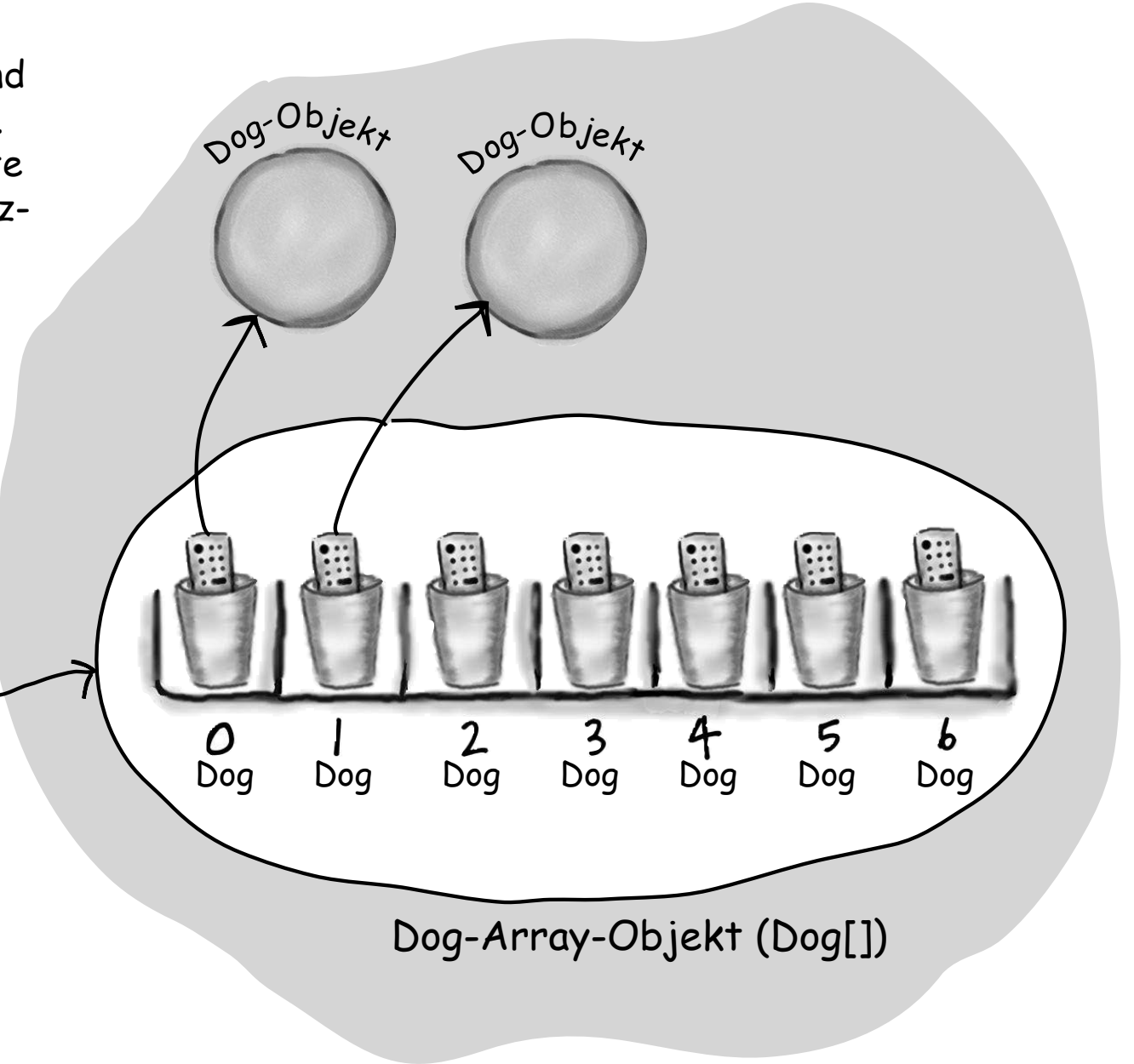
- 3 Neue Dog-Objekte erstellen und den Array-Elementen zuweisen. Beachten Sie, dass die Elemente im Dog-Array nur Dog-Referenzvariablen sind. Wir brauchen immer noch die Dog-Objekte.

`pets[0] = new Dog();`
`pets[1] = new Dog();`

→ Das kriegen Sie selbst hin.



Wie lautet der aktuelle Wert von `pets[2]`? _____
Welcher Code würde `pets[3]` mit einem der vorhandenen Dog-Objekte verknüpfen?



```
1 public class LockerWithObjectsDemo {
2     public static void main(String[] args) {
3         System.out.println("-> Control via reference variable:");
4
5         Car mustang = new Car();
6         mustang.sound = "Loud cracks and pops!";
7
8         // use *variable* name and *dot* operator
9         mustang.startEngine();
10
11        System.out.println("-> Control via array:");
12
13        Car garage[] = new Car[3];
14
15        // use *array* name and *index* operator
16        garage[0] = mustang;
17
18        garage[1] = new Car();
19        garage[1].sound = "Whirr...";
20        garage[1].startEngine();
21
22        garage[0].startEngine();
23
24        // What would happen if we were to put a dog in the garage?
25        // garage[2] = new Dog();
26    }
27 }
28
```

STDIN

Input for the program (Optional)

Output:

```
-> Control via reference variable:
Loud cracks and pops!
-> Control via array:
Whirr...
Loud cracks and pops!
```

! ? Quiz

Arrays

- 🤔 Ich mache einige Aussagen – sind diese Aussagen richtig oder falsch?
- 👍 Bei **richtigen** Aussagen **stehen** Sie **kurz auf**!
- 👎 Bei **falschen** Aussagen **bleiben** Sie **sitzen**!



Arrays ermöglichen einen schnellen und frei wählbaren Zugriff auf jedes enthaltene Element über eine Indexposition.

Richtig!

Das vierte Element des Arrays `numbers` hat den Index 3 und wird durch `numbers[3]` adressiert.

Richtig!

Ein Array, das primitive Daten enthält, z. B. `int`, ist selbst kein Objekt, sondern ein elementares Array.

Das Array selbst ist immer ein Objekt, auch wenn es ein Array elementarer Datentypen ist.

Die Elemente eines `int`-Arrays sind einfach `int`-Variablen.

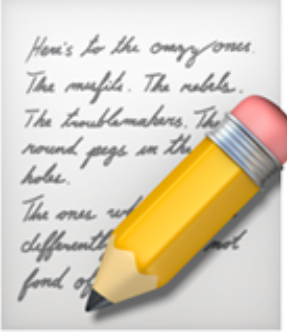
Richtig!

In einem Dog-Array stellt jedes Element eine Referenzvariable zu einem Dog-Objekt dar.

Richtig!

Ein mit `Dog[] pets; pets = new Dog[7];`
erzeugtes Array enthält automatisch 7 Dog-
Objekte.

Nein, jedes Element enthält null.

 Welcher **eine** Gedanke zu
„ Arrays“ war für Sie am
wichtigsten?  Schreiben Sie
ihn auf!

 Durchatmen und  Sortieren



Übungen

Ist der Code fehlerfrei?

Übungen

```
class Books {
    String title;
    String author;
}

class BooksTestDrive {
    public static void main(String[] args) {
        Books[] myBooks = new Books[3];
        int x = 0;
        myBooks[0].title = "The Grapes of Java";
        myBooks[1].title = "The Java Gatsby";
        myBooks[2].title = "The Java Cookbook";
        myBooks[0].author = "bob";
        myBooks[1].author = "sue";
        myBooks[2].author = "ian";
        while (x < 3) {
            System.out.print(myBooks[x].title);
            System.out.print(" by ");
            System.out.println(myBooks[x].author);
            x = x + 1;
        }
    }
}
```

Wird dieser Java-Code
fehlerfrei kompiliert und
ohne Fehler ausgeführt?

Falls nicht, wie
würden Sie den
Code reparieren?

Ist der Code fehlerfrei?

Übungen

```
class Books {
    String title;
    String author;
}
class BooksTestDrive {
    public static void main(String[] args) {
        Books[] myBooks = new Books[3];
        int x = 0;
        myBooks[0].title = "The Grapes of Java";
        myBooks[1].title = "The Java Gatsby";
        myBooks[2].title = "The Java Cookbook";
        myBooks[0].author = "bob";
        myBooks[1].author = "sue";
        myBooks[2].author = "ian";
        while (x < 3) {
            System.out.print(myBooks[x].title);
            System.out.print(" by ");
            System.out.println(myBooks[x].author);
            x = x + 1;
        }
    }
}
```



```
myBooks[0] = new Books();
myBooks[1] = new Books();
myBooks[2] = new Books();
```

Erst die Book-
Objekte erstellen!



Ist der Code fehlerfrei?

Übungen

```
class Hobbits {  
    String name;  
    public static void main(String[] args) {  
        Hobbits[] h = new Hobbits[3];  
        int z = 0;  
        while (z < 4) {  
            z = z + 1;  
            h[z] = new Hobbits();  
            h[z].name = "bilbo";  
            if (z == 1) {  
                h[z].name = "frodo";  
            }  
            if (z == 2) {  
                h[z].name = "sam";  
            }  
            System.out.print(h[z].name + " is a ");  
            System.out.println("good Hobbit name");  
        }  
    }  
}
```

Wird dieser Java-Code
fehlerfrei kompiliert und
ohne Fehler ausgeführt?

Falls nicht, wie
würden Sie den
Code reparieren?

Ist der Code fehlerfrei?

Übungen

```
class Hobbits {  
    String name;  
    public static void main(String[] args) {  
        Hobbits[] h = new Hobbits[3];  
        int z = -1;  
        while (z < 2) {  
            z = z + 1;  
            h[z] = new Hobbits();  
            h[z].name = "bilbo";  
            if (z == 1) {  
                h[z].name = "frodo";  
            }  
            if (z == 2) {  
                h[z].name = "sam";  
            }  
            System.out.print(h[z].name + " is a ");  
            System.out.println("good Hobbit name");  
        }  
    }  
}
```

Der Index des ersten
Array-Elements ist 0!



Pair Programming

🎓 Referenzen und Arrays

🤝 Pair Programming

- 🎯 Erstellen Sie eine Klasse und greifen Sie über ein Array auf deren Instanzen zu.
- ✅ Erstellen Sie eine sehr einfache Klasse wie Book, Car oder Dog.
 - ✅ Zum Beispiel die Klasse Book mit `title` und `printTitle()`.
- ✅ Kontrollieren Sie ein Objekt Ihrer Klasse mit dem Punktoperator.
- ✅ Erstellen Sie ein Array vom Typ Ihrer Klasse.



Referenzen und Arrays

Pair Programming

 **Erstellen Sie eine Klasse und greifen Sie über ein Array auf deren Instanzen zu.**

 Erstellen Sie eine sehr einfache Klasse wie Book, Car oder Dog.

 Zum Beispiel die Klasse Book mit `title` und `printTitle()`.

 Kontrollieren Sie ein Objekt Ihrer Klasse mit dem Punktoperator.

 Erstellen Sie ein Array vom Typ Ihrer Klasse.

 Kontrollieren Sie eine Reihe von Objekten Ihrer Klasse im Array mit dem Indexoperator.

 Bonus: Verwenden Sie eine `while`-Schleife, um Ihr Array zu durchlaufen und den jeweiligen Inhalt einer Instanzvariablen auszugeben.

 Am Ende sollten Sie ein Array mit Instanzen Ihrer Klasse haben, über das Sie diese Instanzen steuern können.


```
1 public class ColoredPencilArrayExampleSolution {
2     public static void main(String[] args) {
3         ColoredPencil redPencil = new ColoredPencil();
4         redPencil.color = "red";
5         System.out.println("Red pencil:");
6         redPencil.draw();
7
8         ColoredPencil[] pencils = new ColoredPencil[3];
9
10        pencils[0] = redPencil;
11
12        pencils[1] = new ColoredPencil();
13        pencils[1].color = "green";
14
15        pencils[2] = new ColoredPencil();
16        pencils[2].color = "blue";
17
18        System.out.println();
19        System.out.println("All " + pencils.length + " pencils:");
20        int index = 0;
21        while (index < pencils.length) {
22            System.out.println("At index: " + index);
23            pencils[index].draw();
24            index = index + 1;
25        }
26    }
27 }
28
```

STDIN

Input for the program (Optional)

Output:

Red pencil:

Drawing in: red

All 3 pencils:

At index: 0

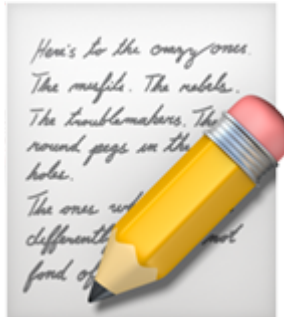
Drawing in: red

At index: 1

Drawing in: green

At index: 2

Drawing in: blue

 Welcher **eine** Gedanke zu
„ Referenzen und Arrays“ war
für Sie am wichtigsten?
 Schreiben Sie ihn auf!

 Durchatmen und  Sortieren

 **Zielgerade**

↔ Teach Back

📖 Elementare Datentypen und Referenzen

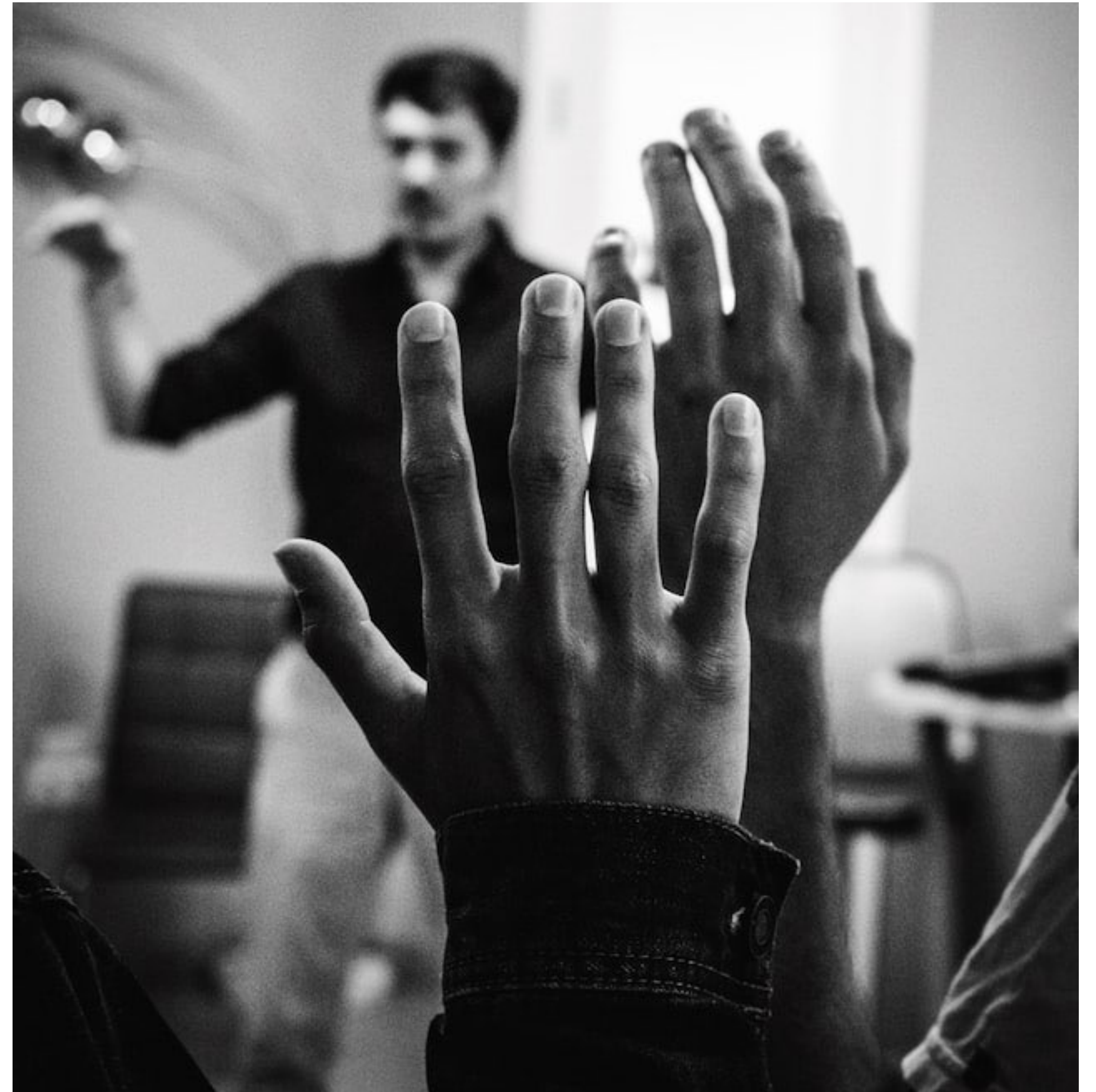
- 🎤 Beantworten Sie sich zusammen mit einer Nachbar:in gegenseitig kurz und knapp die folgenden Fragen:
- 💡 Was habe ich heute gelernt und erfahren?
- 💡 Was davon ist für mich besonders wichtig?



💡 Ihre Fragen

👉 Unsere Antworten

- Welche Fragen haben Sie zu  **Elementare Datentypen und Referenzen?**
- Was irritiert Sie vielleicht sogar?
- Was interessiert Sie außerdem?





Lern-Logbuch



Elementare Datentypen und Referenzen



Machen Sie sich Notizen:



Was sind für mich die wichtigsten Erkenntnisse?



Was weiß ich jetzt, was ich vorher nicht wusste?



Wie kann ich dieses Wissen anwenden?



Welche Fragen habe ich noch?



Was werde ich tun, um sie zu beantworten?



Ihr Lernerfolg – Unser Applaus

